

软件安全两套考试卷汇总

一、第一套试卷选填

1. 函数栈帧是系统实现函数调用过程中需要维护的基本数据结构,函数栈帧的基本信息包括:_____, _____, _____ 和 _____。
2. 操作系统中进程所使用的内存根据用途不同,按照功能主要可分成 _____, _____, _____ 和 _____ 四大部分。
3. 常见的字符串操作错误主要包括 _____, _____, _____, _____, _____ 等。
4. Fuzz 测试的主要目的包括: _____, _____ 和 _____。
5. 采用补码表示,一个 8 比特位无符号整数可以表示的范围是 _____ — _____, 一个 8 比特位有符号整数可以表示的范围是 _____ — _____。
6. 下列无符号类型转换中,会造成数据截断的是 ()
A. unsigned char short
B. unsigned char long
C. unsigned short short
D. unsigned short long
7. 下列对于如下代码,为了使 foo 中取到的地址为 0x10402010, _____ 应该填入 ()。

```
printf("%16u%n%16u%n%32u%n%u%n",  
      (int *)&foo[0], 1, (int *)&foo[1],  
      (int *)&foo[2], 1, (int *)&foo[3]);
```

- A. 224 B. 208 C. 64 D. 128
8. 关于标准 C 语言字符串,以下表述中,正确的是 ()。
A. 由一个连续的字符序列组成,并以一个空字符 (NULL) 作为结束
B. 一个指向字符串的指针实际上就是指向该字符串起始字符
C. 字符串长度是指字符串实际所占存储空间的字节数,含空字符,不含字符串之后的区域
D. 字符串的值是指它所包含的按照顺序排列的字符序列
9. gets()、puts()、strcpy()、strcat()、strlen()、printf()、memcpy() 和 memset() 等 API 均来自标准 C,其中能够导致无界字符串越界写操作的有 () 个。
A. 3 B. 4 C. 5 D. 6
10. 下列 Windows 机制中不是针对缓冲区溢出的技术为 ()。
A. GS 编译技术 B. DEP C. ASLR D. S.E.H
11. 函数调用过程中,指向栈顶和栈底的指针会发生变化。如图的指令组合通常发生在 ()。

```
1. push ebp  
2. mov ebp, esp  
3. sub esp, 28  
...
```

- A. 函数调用过程
- B. 被调用函数的开头部分
- C. 被调用函数的结尾部分
- D. 函数中控制流转执行过程

12. 对于如下代码的执行结果：

```
char c;
printf("%100u%n", 1, &c);
```

c 的值用十六进制表示为 ()。

- A. 0x01 B. 0x64 C. 0x100 D. 0x04

13. 缓冲区溢出能够用于覆盖写函数或数据指针，必须 () 同时成立。

- A. (1)、(2)、(4)
- B. (1)、(2)、(5)
- C. (1)、(2)、(3)、(4)
- D. (1)、(2)、(3)、(5)

(1) 缓冲区与目标函数或数据指针必须分配在同一个段内

(2) 缓冲区必须可以被缓冲区溢出利用

(3) 缓冲区的数据类型必须与目标指针的类型相同

(4) 缓冲区必须位于目标函数或数据指针更低的地址处

(5) 缓冲区必须位于目标函数或数据指针更高的地址处

14. 函数栈帧是函数调用过程中使用的重要数据结构，函数栈帧包含的数据内容有 ()。

- A. (1)、(2)
- B. (1)、(2)、(3)
- C. (1)、(2)、(3)、(4)
- D. (1)、(2)、(3)、(4)、(5)

(1) 调用者栈帧信息；

(2) 函数参数信息；

(3) 函数局部变量；

(4) 函数静态变量；

(5) 函数全局变量。

15. 在下列程序中如果采用栈重定位内存写，则以下说法错误的是 ()。

```
void foo(void *arg, size_t len) {
    char buff[100];
    long val = ...;
    long *ptr = ...;
    memcpy(buff, arg, len);
    *ptr = val;
    ...
    return;
}
```

- A. 第 3 行在溢出缓冲区后，攻击者可以覆盖 ptr 和 val
- B. 第 4 行的指针接下来被用作一个赋值操作的目的地址
- C. 第 5 行并不会导致溢出
- D. 第 6 行会发生任意内存写

16. 运行以下语句会产生什么情况（ ）。

```
printf("%s%s%s%s%s%s%s");
```

- A. 输出 printf 栈帧中的参数字符串
- B. 输出无效指针存放或未映射的地址读取
- C. 输出从 printf 栈底开始的多个字符串
- D. 不输出任何内容

17. 以下程序预期的打印结果有（ ）种情况。

```
int main(int argc, char* argv[]) {
    char source[10];
    strcpy(source, "0123456789");
    char *dest = (char *)malloc(strlen(source));
    for (int i = 1; i <= 11; i++) {
        dest[i] = source[i];
    }
    dest[i] = '\0';
    printf("dest=%s", dest);
}
```

- A. 1
- B. 2
- C. 3
- D. 不确定

18. 以下程序预期的输出结果是（ ）。

```
signed char c1, c2, c;
c1 = 120;
c2 = 120;
c = c1 + c2;
printf("%d", c);
```

- A. 240
- B. -16
- C. -15
- D. -113

19. 为了防止整数的溢出，一般需要进行先验条件测试，以下对于先验条件的叙述错误的是（ ）。

- A. 将两个操作数放到下一个更大的数据类型上，然后相乘
- B. 对于无符号整数检查一个大整数的商和值，如果溢出了，则检查错误
- C. 对于带符号整数，如果结果的高半部分和低半部分的符号位全为 0 或 1，则没有发生整数溢出
- D. 为了防止无符号整数相乘时发生溢出，检验 $A * B \geq \text{MAX_INT}$ 是否成立

20. 下列关于地址的随机化，有误的一项是（ ）。

- A. 地址的随机化可以使得 malloc() 等函数每次返回的地址是随机的，这让程序每次展现出不同的空间行为
- B. 随机化会让程序的调试工作更加困难
- C. 地址随机化可以带来更小的性能开销和更好的运行效果
- D. 地址随机化使得漏洞利用更加困难

第一套试卷选填参考答案

1. 参数、局部变量、栈帧状态值、函数返回地址

2. 代码区、数据区、堆区、栈区
3. 无边界字符串复制、空结尾错误、字符串截断、差一错误、数组写入越界
4. 崩溃、中断、销毁
5. 0, 255, -128, 127
- 6.C 7.B 8.C 9.C 10.D 11.B 12.B 13.A 14.B 15.C 16.B 17.C 18.B 19.D
20.C

二、第二套试卷选填

- (1-1) 侦测整数错误的方法主要有 _____ 和 _____。
- (1-2) 采用补码表示，一个 8 比特的数据，无符号整数可以表示的范围是 _____，有符号整数可以表示的范围是 _____。
- (1-3) 操作系统中进程所使用的内存根据用途不同,按照功能主要可分成 _____, _____, _____ 和 _____ 四大部分。
- (1-4) 常见的字符串操作错误主要包括: _____ 等。
- (1-5) 一致的内存管理的定义包括 _____ 等。
- (2-1) 如下表赋值表达式所示，names 是一个 UTF-8 的字符串：

```
char names[] = "\xD4\xBC\xF0\x9D\x90\x84\x45\xC6\xAC\x00";
```

那么该字符串的长度是 ()。

- A. 4 B. 10 C. 9 D. 5

(2-2) 下列 Windows 机制中不是针对缓冲区溢出的技术为 ()。

- A. GS 编译技术
B. S.E.H
C. ASLR (Address space layout randomization)
D. DEP

(2-3) 该段代码第三行该如何为变量 prog_name 分配内存? ()

```
int main(int argc, char *argv[]) {
    const char *const name = argv[0] ? argv[0] : "";
    // 补充: 声明 prog_name 并分配内存
    if (prog_name != NULL) {
        strcpy(prog_name, name);
    }
}
```

- A. char *prog_name = malloc(strlen(name));
B. char *prog_name = malloc(strlen(name) + 1);
C. char *prog_name = (char *)malloc(strlen(name));
D. char *prog_name = (char *)malloc(strlen(name) + 1);

(2-4) gets()、puts()、strcpy()、strcat()、strlen()、printf()、memcpy() 和 memset() 等 8 个 API 均来自标准 C，其中能够导致无界字符串越界写操作的有 () 个。

A. 3 B. 4 C. 5 D. 6

(2-5) 根据以下代码，任何长度大于（ ）字节的字符串都会导致溢出：

```
char buffer[512];
sprintf(buffer, "Wrong command: %s\n", user);
```

A. 495 B. 494 C. 493 D. 492

(2-6) 关于异常处理，以下哪种说法是错误的（ ）。

- A. 栈探测可以保护包括栈段在内的任何位置发生缓冲区溢出
- B. 攻击者可以通过覆盖异常处理程序地址修改指令指针
- C. 在 try ...catch 块中，如果 catch 块无法处理异常，那么会将捕获之前的栈展开
- D. 已注册的异常处理程序列表是由 Thread Environment Block (TEB) 中的指针引用的

(2-7) 缓冲区溢出能被用于覆写函数或数据指针，必须（ ）同时成立。

- a. 缓冲区与目标函数或数据指针必须分配在同一个段内；
- b. 缓冲区必须可以被缓冲区溢出利用；
- c. 缓冲区的数据类型必须与目标指针的类型相同；
- d. 缓冲区必须位于目标函数或数据指针更低的地址处；
- e. 缓冲区必须位于目标函数或数据指针更高的地址处。

A. a, b, c, d

B. a, b, e

C. a, b, d

D. a, b, c, e

(2-8) 考虑下面的代码：

```
enum { BLOCK_HEADER_SIZE = 16 };

void *AllocateBlock(size_t length) {
    struct memBlock *mBlock;
    if (length + BLOCK_HEADER_SIZE > (unsigned long long)SIZE_MAX) {
        return NULL;
    }
    mBlock = (struct memBlock *)malloc(
        length + BLOCK_HEADER_SIZE
    );
    if (!mBlock) return NULL;
    /* fill in block header and return data portion */
    return mBlock;
}
```

该程序最根本的漏洞是什么（ ）。

- A. 非异常整数逻辑错误
- B. 有符号整数溢出
- C. 转换错误
- D. 无符号整数回绕

(2-9) 关于限制字符串输入的漏洞缓解方法，下列说法错误的是（ ）。

- A. 缓冲区溢出可以通过严格控制输入的字节数来避免
- B. snprintf() 比 sprintf() 更安全，vsprintf() 比 vsprintf() 更安全

- C. 函数 `asprintf()` 和 `vasprintf()` 为字符串分配足够大的空间以容纳包括末尾空字符在内的输出内容
- D. 使用更安全版本的格式化输出库函数可以杜绝缓冲区溢出问题
- (2-10) 要成功利用双重释放漏洞, 需满足的条件不包括 ()。
- A. 被释放的内存块必须在内存中独立存在
- B. 被释放的内存块相邻的内存块必须是已分配的
- C. 被释放的内存块相邻的内存块必须是未分配的
- D. 被释放的内存块所被放入的 bin 必须为空

第二套试卷选填参考答案

1. 奇偶校验、循环冗余校验

2. $0 \sim 2^n - 1$, $-2^{n-1} \sim 2^{n-1} - 1$

3. 代码区、数据区、堆区、栈区

4. 无边界字符串复制、空结尾错误、字符串截断、差一错误、数组写入越界

5. 页式存储管理、段式存储管理

1.A 2.B 3.D 4.C 5.A 6.A 7.C 8.D 9.D 10.C

三、简答题

1. 对比植入 shellcode 中静态硬编码地址与 JMP ESP 两种方式的优缺点。

答题区

静态淹没返回地址: 用大量重复的返回地址覆盖栈上返回地址区域 (喷射/淹没), 通常配合 `NOP sled` 扩大命中。

优点: 实现简单、对偏移误差容忍较高 (可通过 `NOP` / 重复地址提高命中)。

缺点: 需要可预测的跳转目标地址; 地址中若含 `0` 字节等可能受字符串函数限制; 遇到栈随机化 / ASLR 难度增加。

JMP ESP 方式: 返回地址覆盖为某模块中的 `JMP ESP` 指令地址, 使 `EIP` 跳到 `ESP` 指向处执行栈上 shellcode。

优点: 不必精确知道 shellcode 绝对地址, 只要能控制栈上内容; 相对短跳转且结构清晰。

缺点：依赖可用的 **JMP ESP** 地址（受模块版本 / ASLR / DEP 影响）；仍可能受坏字符限制（写返回地址时）。

2. 解释为什么格式化输出可能导致程序崩溃？

答题区

- **读越界 / 栈参数错位**：格式化输出函数（如 printf）属于变参函数，函数在运行时不会检查参数的数量和类型是否与格式字符串匹配。当格式字符串被错误或恶意构造时，printf 会按照格式串从栈中依次取参数，导致栈参数错随机内容并将其当作地址使用，从而访问非法内存并引发程序崩溃。
- 此外，格式符 %n 会将已输出的字符数写入指定地址，若该地址非法或不可写，也会导致程序异常终止。

3. 简述软件漏洞能够导致的后果有哪些？

答题区

PPT 中给出的后果包括：无法正常使用；引发恶性事件；关键数据丢失；秘密信息泄露；被安装木马病毒。

综合分析还包括：程序崩溃或拒绝服务（DoS）；信息泄露（如格式化字符串漏洞读栈或任意地址）；权限提升与执行任意代码（缓冲区溢出、%n 任意写等）；数据破坏与业务损失。

4. 指出并分析下图程序的整数溢出问题。

```
void getComment(unsigned int len, char *src) {
    unsigned int size;
    size = len - 2;
    char *comment = (char *)malloc(size + 1);
    memcpy(comment, src, size);
    return;
}

int _tmain(int argc, _TCHAR* argv[]) {
    getComment(1, "Comment");
    return 0;
}
```

答题区

- (1) 首先, 变量 `size` 是 `unsigned int`。执行 `size = len - 2` 时, 若 `len < 2`, 数学结果为负数, 但无符号类型不能表示负数, 因此发生回绕。例如 `len = 1` 时, `size` 变为 `0xFFFFFFFF`。
 - (2) 随后执行 `malloc(size + 1)`, 此时将基于一个错误的巨大 `size` 值进行内存分配, 可能导致内存分配异常或分配到不合理的内存空间。程序未对 `malloc` 的返回值进行检查, 紧接着调用 `memcpy(comment, src, size)`, 会尝试按照 `0xFFFFFFFF` 的长度进行拷贝, 从而造成内存访问越界, 破坏其他内存区域, 最终引发程序崩溃。
 - (3) 在 `getComment(1, "Comment")` 的调用中, 当注释长度字段 `len = 1` 时即满足 `len < 2`, 从而触发上述整数溢出问题, `size` 变为 `0xFFFFFFFF`, 导致段错误 (缓冲区溢出 / 内存访问越界)。
5. 不安全的 API 是导致字符串错误的重要原因, 试列举并说明 5 个不安全的字符串 API?

答题区

- `strcpy(dst, src)`: 不检查长度, `src` 过长直接溢出。
- `strcat(dst, src)`: 不检查剩余空间, 拼接可能越界。
- `sprintf(buf, fmt, ...)`: 不限制输出长度, 格式化后可能写爆 `buf`。
- `gets(buf)`: 读取一行不做边界检查 (经典栈溢出)。
- `scanf("%s", buf)`: 未限制宽度时可导致溢出 (应使用 `%Ns`)。

6. 指出下列代码的错误。

```
/* return y = A x */
int *matvec(int **A, int *x, int n) {
    int *y = malloc(n * sizeof(int));
    int i, j;
```



```

    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            y[i] += A[i][j] * x[j];
    return y;
}

```

答题区

- (1) `int *y = malloc(n * sizeof(int));`: `malloc` 分配的内存未初始化, `y[i]` 直接使用 `+=` 会在随机初值上累加, 结果未定义。应先将 `y` 清零 (如使用 `calloc`, 或循环前令 `y[i]=0`)。
- (2) 未检查 `malloc` 是否成功 (是否返回 `NULL`), 失败时继续访问会导致崩溃。
- (3) `n * sizeof(int)` 可能整数溢出, 导致分配空间不足并引发越界写。

7. 根据输入的不同, 列举并分析下图程序可能的执行结果。

```

bool IsPasswordOkay(void) {
    char Password[12];
    gets(Password);
    if (!strcmp(Password, "goodpass"))
        return(true);
    else
        return(false);
}

void main(void) {
    bool PwStatus;
    puts("Enter password:");
    PwStatus = IsPasswordOkay();
    if (PwStatus == false) {
        puts("Access denied");
        exit(-1);
    }
    else
        puts("Access granted");
}

```

答题区

- (1) 输入 "goodpass": `strcmp` 返回 0, 取反为真, 函数返回 true, 输出 "Access granted".
- (2) 输入其他错误密码 (长度不超过 11, 且不等于 "goodpass"): `strcmp` 不为 0, 返回 false, 输出 "Access denied" 并退出。
- (3) 输入长度恰好为 11 的字符串: 由于 11 个字符加结尾空字符正好占 12 字节, 不会发生越界; 但不可能等于 "goodpass", 因此仍输出 "Access denied".
- (4) 输入长度至少为 12 的字符串: `gets` 不做边界检查, 会写入超过 `Password[12]` 的容量, 发生栈缓冲区溢出。可能导致崩溃 (段错误)、异常行为, 甚至破坏栈帧/控制流造成绕过认证或被进一步利用。

8. 简述 Fuzz 测试的思想, 并描述针对文件的 Fuzz 测试的步骤和流程。

答题区

技术思想: 通过利用暴力来实现对目标程序的自动化测试, 然后监视检查其最后的结果, 如果符合某种情况就认为程序可能存在某种漏洞或者问题。暴力是利用不断地向目标程序发送或者传递不同格式的数据来测试目标程序的反应。

文件格式 Fuzz 测试流程:

1. 以一个正常的文件模板作为基础, 按照一定规则产生一批畸形文件;
2. 将畸形文件逐一送入软件进行解析, 并监视软件是否会抛出异常;
3. 记录软件产生的错误信息, 如寄存器状态、栈状态等;
4. 用日志或其他 UI 形式向测试人员展示异常信息, 以进一步鉴定这些错误是否能被利用。

9. 列举动态分配缓冲区的缺点。

答题区

- (1) **易出现堆相关漏洞**：使用后释放（UAF）、双重释放、堆溢出破坏链表指针等，利用面更复杂但后果严重。
- (2) **需要理解并依赖堆管理数据结构**（如空闲链表、边界标志等），错误使用内存管理 API 可能引入漏洞。
- (3) 在不同系统 / 版本下**堆布局与分配策略差异大**，漏洞复现与稳定利用更困难。环境依赖强、调试复杂、易受缓解策略影响。
- (4) **可能导致内存耗尽**：若分配大小由外部输入控制，攻击者可通过构造输入反复申请大量内存，造成拒绝服务（DoS）。
- (5) **性能开销较大**：动态分配涉及堆管理、空闲块查找及合并等操作，相比静态分配效率较低。

10. 简述虚函数的实现，并试述利用虚表对虚函数展开攻击的过程。假设声明了一个类，具有 160 字节的成员变量 buf 和虚函数 test，main 函数中可以通过溢出修改虚表，指出做法。

答题区

实现机制：

- (1) 编译器为含虚函数的类生成虚函数表 **VTBL**（函数指针数组）。

- (2) 每个对象内存起始处保存虚表指针 **VPTR**，调用虚函数时先取 VPTR 定位 VTBL，再取对应槽位的函数指针执行，实现运行时动态派发。

攻击思路（溢出改 **VPTR** / 改 **VTBL** 条目）：已知对象布局：[VPTR][buf(160 bytes)]

- (1) 方式 A：改 **VPTR**。攻击者通过溢出覆盖对象头部的 VPTR，使其指向攻击者伪造的 VTBL（可放在 buf 里或可写内存中）。当 `p->test()` 调用发生，程序从伪 VTBL 取到伪造的函数指针（指向 shellcode 或 ROP 链），控制流被劫持。
- (2) 方式 B：改 **VTBL** 中的函数指针。若能写到真实 VTBL 所在内存（或借助任意写），可直接把 VTBL 里 test 入口改成 shellcode 地址；之后任何对象调用该虚函数都会跳转到恶意地址。

关键点：虚函数动态调用必须“通过对象指针 / 引用”触发，攻击者目标就是让最终取出的函数入口地址变为攻击者控制地址。

11. 指出下列代码在什么情况下，就会发生缓冲区溢出。

```
void good_function(const char *str) { ... }

void main(int argc, char **argv) {
    static char buff[BUFSIZE];
    static void (*funcPtr)(const char *str);

    funcPtr = &good_function;
    strcpy(buff, argv[1], strlen(argv[1]));
    (void)(*funcPtr)(argv[2]);
}
```

答题区

当 `strlen(argv[1]) ≥ BUFSIZE` 时，将 `argv[1]` 复制到 `buff[BUFSIZE]` 中无法容纳完整字符串及其结尾 `\0`，因此发生缓冲区溢出。

12. 请解释 `printf("%s%s%s%s%s%s%s%s%s%s")` 格式化输出可能导致程序崩溃？

答题区

`printf` 是变参函数，其参数数量和类型完全由格式字符串决定，但函数在运行时不会对实参个数进行检查。

在该语句中，格式字符串包含多个 `%s`，而调用时并未提供对应的字符串参数。执行过程中，`printf` 会按照格式串的要求，从栈中依次取出“参数”并将其解释为字符串指针。

由于这些栈上的值并非有效的指针，当 `%s` 试图按照该指针访问内存并输出字符串时，可能解引用非法或未映射的地址，从而触发段错误，导致程序崩溃。

13. 什么是空闲内存列表，并画出其关键数据结构。

答题区

空闲内存列表是堆管理器用于记录并组织空闲堆块的数据结构。当调用 `free` 释放内存时，空闲块会被插入空闲链表；当调用 `malloc/HeapAlloc` 分配内存时，管理器会在空闲链表中查找合适大小的空闲块并将其取出或分裂。

在 Windows RTL Heap 中，空闲块通常通过多个双向链表按大小类别组织（可表示为 `Freelist[]` 的链表头数组），每个链表头为 `LIST_ENTRY`：

```
Freelist[i] : LIST_ENTRY { Flink, Blink }
```

```
    Flink --> next free chunk
```

```
    Blink --> prev free chunk
```

FreeChunk:

```
+-----+
| ... header / size ... |
| LIST_ENTRY: Flink Blink |
| (free space ...)      |
+-----+
```

当某个链表为空时，该链表头的 `Flink` 与 `Blink` 均指向链表头自身（自环）。

14. 先写出 `frontlink` 技术实现代码，分析它并给出实现攻击者提供 4 字节的数据写入到同样是攻击者指定的 4 字节地址的实例详细过程。

答题区

```

int _tmain(int argc, _TCHAR* argv[]) {
    Punalloc h1;
    HLOCAL h2 = 0;
    HANDLE hp;
    hp = HeapCreate(0, 0x1000, 0x10000);
    h1 = (Punalloc)HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
    HeapFree(hp, 0, h1); // 错误释放
    h1->fp = (PVOID)(0x042B17C - 4); // 目标地址-4
    h1->bp = shellcode; // 要写入的 4 字节值
    h2 = HeapAlloc(hp, HEAP_ZERO_MEMORY, 32);
    HeapFree(hp, 0, h2); // 控制转移到 shellcode
    return 0;
}

```

利用空闲链表指针（flink / blink）实现任意写的解释：释放后再写入已释放块的 fp/bp（等价 flink / blink），通过后续分配 / 释放触发链表操作，完成覆盖目标地址为 shellcode 地址，从而劫持控制流。

4 字节任意写过程（写“值 = bp”到“地址 = fp + 4”）：

- (1) h1 被释放后进入对应大小的空闲链表；空闲块用户区前两个 DWORD 保存链表指针（flink / blink）。
- (2) 漏洞点：对已释放的 h1 写入，将 fp 伪造为“待改写地址 - 4”，将 bp 伪造为“攻击者希望写入的 4 字节值（如 shellcode 地址）”。
- (3) 下一次 HeapAlloc 拿到该空闲块时，RtlHeap 把块从双向链表摘除，会执行类似：fp->bk = bp; bp->fd = fp; 的指针更新，从而把 bp 写到 fp+4 处，实现“向任意地址写入 4 字节”。材料明确指出：这次 HeapAlloc 导致“HeapFree() 地址被 shellcode 地址覆盖”。
- (4) 随后触发一次调用（例中再次 HeapFree），使控制流跳到被覆盖的地址（shellcode）。